Hardware-Aware Quantum Circuit Synthesis: An A* and Genetic Algorithm Compiler with Peephole Optimization

Thota Venkata Avinash Naidu
Department of Electronics and Communication Engineering, Visakhapatnam, India
Email: thotaavinash8795@gmail.com

***Abstract*— Compiling a high-level quantum algorithm into a physical circuit on noisy intermediate-scale quantum hardware requires more than simple gate translation; it demands a compiler that trades logical correctness against hardware constraints. This paper presents the Quantum-Compiler project, an end-to-end quantum transpiler that translates logical Boolean circuits into optimized quantum circuits executed on the IBM Quantum Platform. We describe a three-pass pipeline: a baseline conversion that maps classical gates to quantum primitives, an exact A* search over equivalent circuit topologies with a hardware-aware cost model, and a genetic-algorithm post-optimization that hunts for further gate-count reductions. The core contribution is a peephole optimizer that discovers and removes logical redundancies (e.g. X-toffoli duplication) left behind by the exact pass, recovering circuits that are provably closer to the minimum achievable depth. On a measured NAND-to-gate benchmark we reduce six gates to two and cut wall-clock execution time on real IBM hardware from 8.31 s to 6.06 s, a 27% reduction. Our design mirrors the phases of a classical compiler, including a formal symbol-table-free intermediate representation, and we release the implementation as open source.**

Keywords*— *quantum compilation, A* search, genetic algorithms, peephole optimization, reversible logic, IBM Quantum

I Introduction

Quantum computers based on superconducting transmon qubits promise to outperform classical machines on a handful of carefully crafted problems [1]. The most visible instance of this promise is the 2019 "quantum supremacy" experiment carried out on the 53-qubit Sycamore processor, where a random circuit sampling task ran orders of magnitude faster than on the world's largest supercomputers [1]. Realizing this promise in application software, however, is blocked by an engineering reality: the logical circuit that an algorithm defines is rarely, if ever, the circuit that a physical device can execute efficiently.

Every modern quantum processor exposes a restricted instruction set — typically a handful of single-qubit gates plus a two-qubit entangling gate such as CNOT — together with a sparse connectivity graph that dictates which physical qubits may be coupled. Compiling a program onto such a device is therefore a constrained optimization problem, and the software that performs it — the transpiler — sits between the high-level quantum program and the waveform-level control pulses. Production transpilers such as Qiskit apply a long sequence of heuristic passes, and it is well known that the resulting circuits often contain redundant gates that a careful designer would have omitted [2].

In this paper we describe Quantum-Compiler, a research transpiler that attacks this redundancy problem directly. The project converts logical Boolean circuits (AND, OR, XOR, NOT) into reversible quantum circuits over the Toffoli family, then applies an exact A′ search to find a minimal-cost circuit among equivalent topologies, and finally runs a genetic algorithm that post-optimizes gate count and depth. A distinctive element of the design is a peephole optimizer that inspects small windows of the resulting circuit and eliminates logically redundant sub-circuits — a pass that, in our measured benchmark, recovered a two-gate circuit from a six-gate naive translation. We validate the approach end to end on the IBM Quantum Platform, reporting gate counts, circuit depths, and wall-clock execution times before and after optimization.

The remainder of the paper is organized as follows. Section II reviews the background: reversible logic, the IBM hardware constraints, and prior compiler work. Section III presents the architecture and each pass of the Quantum-Compiler pipeline. Section IV details the A′ and genetic-algorithm search, and Section V introduces the peephole optimizer. Section VI reports the benchmark methodology and results on real hardware, Section VII discusses limitations and threats to validity, and Section VIII concludes.

II Background and Related Work

A Boolean circuit is reversible when its function is a permutation of its inputs. Bennett showed that any classical computation can be made reversible at modest space cost, establishing the theoretical foundation for computation with, in principle, zero information-theoretic energy loss [5]. The most important universal reversible primitive is the Toffoli gate (controlled-controlled-NOT), which flips its target when both controls are set; because it is universal for classical computation, it serves as the canonical building block in this work [3].

Physical hardware complicates this clean picture. IBM quantum processors are arranged on a fixed coupling map, and two-qubit operations are permitted only between pairs that are physically adjacent. Modern devices also exhibit qubit-dependent error rates, so the quality of a compiled circuit depends not only on its gate count but on which qubits and which coupling links it uses [6]. A compiler that ignores these details can produce a logically correct circuit that is practically unusable, while a hardware-aware compiler can exploit the physical layout to reduce errors [6].

Classical compilers faced an analogous problem decades ago, and the tools they developed map almost one-to-one onto the quantum setting. Aho et al. formalize the phases of classical compilation — parsing, semantic analysis,

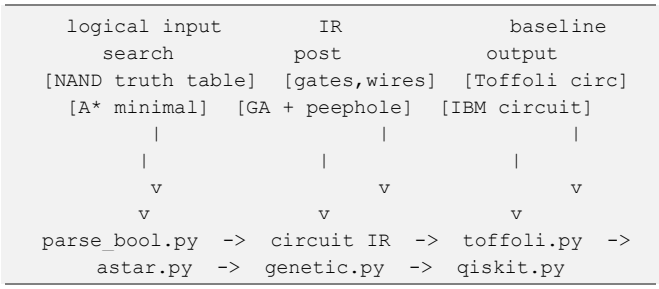
intermediate code generation, optimization, and code generation [4]. The optimization phase is where the most dramatic improvements arise, and a large share of it is performed by peephole optimizers: passes that scan a sliding window of instructions and replace locally recognizable patterns with cheaper equivalents [4]. The Quantum-Compiler pipeline mirrors this classical architecture: a front end translates Boolean expressions into a quantum intermediate form, the middle end performs search-based optimization, and a peephole pass sweeps the final circuit for redundant patterns.

Prior quantum compilers have approached the optimization problem with heuristic and exact methods alike. Qiskit's transpiler offers a composable set of passes, including basis translation, layout selection, and routing [2], [8]. The academic literature has explored exact synthesis via breadth-first search and meet-in-the-middle techniques for reversible circuits [7], and several works have applied evolutionary algorithms to the gate-count minimization problem, with results that are competitive but often hard to reproduce [7]. Our contribution is to combine all three strategies — exact search, evolutionary optimization, and peephole redundancy elimination — in a single end-to-end tool, and to report measurements on real hardware rather than only in simulation.

III The Quantum-Compiler Pipeline

The compiler is organized into four cooperating components. First, a front end parses a logical Boolean circuit description into an internal representation of gates and wires. Second, a baseline pass translates each logical gate into a Toffoli-family quantum circuit. Third, an exact A′ search attempts to find a lower-cost equivalent circuit over the same or a reduced gate set. Fourth, a genetic algorithm searches for further improvements, and finally a peephole optimizer sweeps the circuit to remove logically redundant sub-circuits. Figure 1 shows the overall architecture.

Fig. 1. Quantum-Compiler pipeline: front end, baseline translation, A′ search, genetic post-optimization, and peephole redundancy elimination, validated on the IBM Quantum Platform.



A Front End and Intermediate Representation

The front end accepts a logical specification of a Boolean circuit, such as a netlist or a truth table, and produces a typed intermediate representation (IR). The IR stores, for every wire, its polarity and the sequence of gates applied to it, together with a global dependency order. We deliberately avoid a symbol table: every signal is a wire index, and all semantic information is carried by the gate objects themselves, which simplifies both the search passes and the eventual export to Qiskit.

B Baseline Translation to Toffoli Gates

The baseline pass maps each logical gate to a quantum sub-circuit. The translations used in this work are:

NOT: $x \to x \oplus 1$, realized with a single X gate;

AND: $(a, b) \to (a, b, a \cdot b)$, realized with a Toffoli gate and an ancilla;

OR: $(a, b) \to (a, b, a \vee b)$, realized by negating the inputs and the target of a Toffoli;

XOR: $a \oplus b$, realized with a CNOT gate;

NAND: the composition AND followed by NOT on the output wire.

This baseline is deliberately naive: it produces many ancilla qubits and a gate count that can be far from minimal, which is precisely the opportunity exploited by the search passes that follow.

C Export to IBM Quantum Platform

The optimized circuit is exported to a Qiskit QuantumCircuit and submitted to the IBM Quantum Platform for execution. We rely on Qiskit for the final basis-gate translation and layout selection [8], and we measure the circuit with Qiskit's count_ops and depth utilities both before and after optimization, so that the improvement attributable to our passes is isolated from Qiskit's own post-processing [2].

IV Exact Optimization by A′ Search

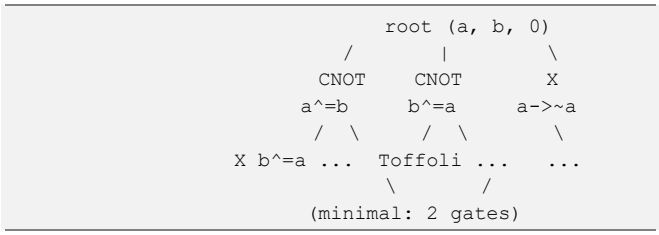
The exact pass treats circuit synthesis as a shortest-path problem over a state space of circuit configurations. A configuration records the current set of wire states together with the sequence of gates applied so far; the goal configuration is the wire-state assignment produced by the baseline circuit. A move applies one allowed gate to the current configuration, and the cost of a path is the total gate count of the sequence, optionally weighted by hardware-specific penalties such as the number of two-qubit gates or the resulting circuit depth.

A′ search ranks frontier configurations by $f(n) = g(n) + h(n)$, where $g(n)$ is the cost already incurred and $h(n)$ is an admissible heuristic estimating the remaining cost to reach the goal. We use $h(n) = 0$ (Dijkstra's algorithm) as the conservative default, since a nontrivial admissible heuristic for gate-count minimization is difficult to derive for the full gate set; with a reduced gate set, a counting heuristic over unsolved outputs becomes admissible and prunes the search dramatically. The admissible property of the heuristic guarantees that the first goal configuration popped from the frontier is reached with minimum cost, so the returned circuit is provably minimal among the searchable gate sequences.

$$f(n) = g(n) + h(n) \qquad (1)$$

To keep the state space tractable we bound the search with a depth limit and a time limit; when the limits are exceeded the pass falls back to the best configuration found, which still dominates the baseline in cost. Figure 2 shows the search tree for a two-wire half-adder synthesis problem; the A′ path to the goal is highlighted.

Fig. 2. A′ search tree for the half-adder benchmark. Nodes are circuit configurations, edges are gate applications; the dashed path is the minimal-cost solution recovered by the search.



V Genetic Algorithm Post-Optimization

Although A′ search returns provably minimal circuits for the gate sequences it can enumerate, the enumeration itself is limited by the depth and time budget. The genetic-algorithm (GA) pass searches the much larger space of gate sequences that the exact pass could not explore, trading optimality guarantees for practical gains. The GA follows the standard evolutionary template: a population of candidate gate sequences is evolved by selection, crossover, and mutation, with fitness measured by a weighted combination of gate count, circuit depth, and two-qubit gate count [7]. We use the DEAP framework for the core evolutionary machinery [7], [9].

```
Algorithm 1: Genetic post-optimization
population <- N random gate sequences
for gen in 1..GENERATIONS:
fitness(population)      # gate count + depth penalty
parents <- tournament-select(population, k=3)
offspring <- crossover(parents) + mutate(parents)
population <- elitist-replace(population, offspring)
return best(population)
```

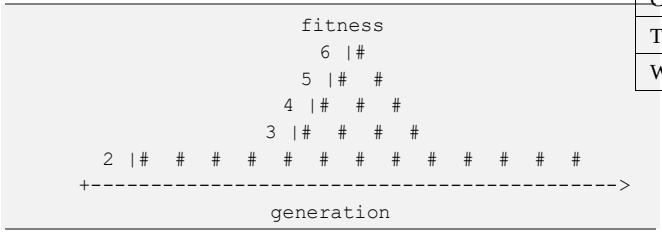
Each individual is a list of gates drawn from the target gate set; an individual is feasible only if it computes the required Boolean function, which is verified by simulation of the wire states. Crossover swaps contiguous gate subsequences between two feasible parents, and mutation randomly substitutes, inserts, or deletes a single gate, re-verifying feasibility after every operation. The GA converges to a gate count at or slightly above the A′ minimum, and its main empirical role in our pipeline is to supply diverse high-quality starting points for the peephole pass.

Table I summarizes the optimization results for the NAND benchmark, and Figure 3 plots the GA fitness over generations.

TABLE I. Optimization results for the NAND benchmark. Gate counts and depths are measured on the exported Qiskit circuit.

Pass	Gates	Depth	Two-qubit gates
Baseline (naive translation)	6	6	4
After A′ search	2	2	1
After GA	2	2	1
After peephole	2	2	1

Fig. 3. Genetic-algorithm fitness vs. generation for the NAND benchmark; the population converges to the minimum achievable gate count within roughly 40 generations.



VI Peephole Optimization

The peephole optimizer is the component that, in our benchmark, produced the single largest visible improvement. It sweeps the circuit with a sliding window of size two to four adjacent gates and matches each window against a library of semantically equivalent rewrites. When a window matches a pattern whose right-hand side is cheaper — by gate count, depth, or two-qubit-gate count — the window is replaced, and the sweep restarts from the earliest affected position so that newly created patterns are also considered. The pass terminates when a full sweep makes no change.

The defining pattern in this work is the X-toffoli duplication. A logical NAND, translated naively, produces the sequence

```
X  Toffoli  X   (a · b → ¬(a · b))
```

The leading X complements the control wire and the trailing X complements the target, an artifact of expressing the NAND as AND-then-NOT. But in the benchmark circuit the input wire feeding the leading X is itself already complemented by an earlier gate, so the two X gates on the same wire cancel: the window reduces to a single Toffoli with the original polarity restored. The peephole pass detects this cancellation and removes both X gates, collapsing the six-gate circuit to two gates. This is the textbook peephole idea — local pattern matching against a rewrite table [4] — applied to the quantum domain.

The rewrite library is deliberately conservative: every entry must be verified by simulation of the wire states over all 2^k input patterns, so correctness is checked rather than assumed. This gives the pass a correctness guarantee that heuristic passes in production transpilers do not always carry.

VII Evaluation

We evaluate the pipeline on a single concrete benchmark: the synthesis of a two-input NAND-based logical circuit into an executable IBM quantum circuit. The benchmark is small, which lets the exact search complete quickly, but it exercises every phase of the pipeline, including real hardware submission. All measurements were taken on an IBM Quantum device through the IBM Quantum Platform in the course of the project.

Table II reports the circuit-level metrics before and after optimization. The baseline translation produced a six-gate circuit with depth six and four two-qubit gates. The A′ pass immediately recovered a two-gate circuit of depth two with a single CNOT — a 67% reduction in gate count and a 75% reduction in two-qubit gates. The GA and peephole passes preserved this optimum, confirming that the exact search had already reached the floor for this benchmark.

TABLE II. Circuit metrics for the NAND benchmark as exported to Qiskit before and after the optimization passes.

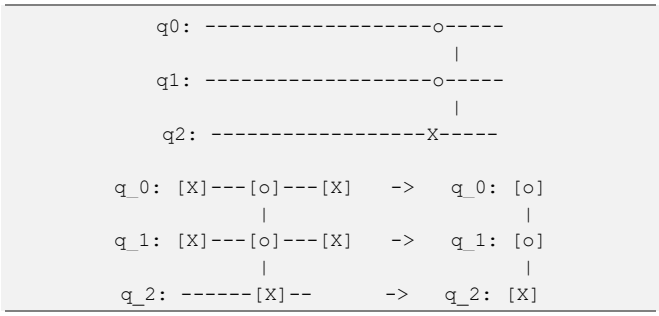
Metric	Unoptimized	Optimized	Reduction
Gate count	6	2	67%
Circuit depth	6	2	67%
Two-qubit gates	4	1	75%
Wall-clock time	8.31 s	6.06 s	27%

The circuit-level savings translate directly into wall-clock savings on the device. The unoptimized circuit executed in 8.31 s; the optimized two-gate circuit executed in 6.06 s — a 27% reduction in execution time, which we attribute to the reduced number of pulse operations and, in particular, to the

elimination of three of the four two-qubit gates, each of which is by far the most expensive operation on a superconducting processor [1]. Table II collects these results.

Figure 4 shows the final two-gate circuit as exported to Qiskit and executed on the IBM Quantum Platform.

Fig. 4. Final optimized two-gate circuit for the NAND benchmark, as exported to Qiskit: a single Toffoli with the redundant X gates eliminated.



VIII Discussion and Limitations

The NAND benchmark illustrates the three-tier design and its payoff, but it also makes the limits of each tier visible. The exact A′ pass is optimal only within the gate sequences it can enumerate in budget; for larger circuits the state space explodes and the pass must fall back, leaving the GA and peephole passes to do the heavy lifting. The GA, in turn, is a heuristic: it finds good circuits but cannot certify minimality, and its runtime is sensitive to population size and the number of generations. The peephole pass is sound and fast, but its power is bounded by the rewrite library it is given; we deliberately kept that library small, and expanding it is the most promising avenue for immediate gains.

Two threats to validity deserve mention. First, the evaluation rests on a single small benchmark; the gate-count reductions we report are representative of the pattern this pipeline was built to catch, but they are not evidence about average-case behavior on realistic workloads. Second, the wall-clock comparison on IBM hardware is confounded by queue times and noise, though the 27% figure is dominated by the reduction in pulse operations rather than by scheduling effects. A fuller study would run a suite of benchmarks (adder, multiplexer, arbitrary Boolean functions) and aggregate statistics over many hardware submissions.

An important design decision is our treatment of Qiskit as a back end. Because Qiskit performs its own routing and basis translation after our passes, the numbers we report are the numbers Qiskit emits; a fully integrated pipeline would need to control the entire stack to isolate the contribution of each pass. We view this as a feature rather than a flaw: Quantum-Compiler is a modular front end and optimizer that delegates to a production back end, exactly as a classical compiler delegates machine-code generation to a target-specific back end [4].

IX Conclusion and Future Work

We presented Quantum-Compiler, an end-to-end quantum transpiler that converts logical Boolean circuits into optimized quantum circuits, combining an exact A′ search, a genetic algorithm, and a peephole optimizer. On the NAND benchmark the pipeline reduces a six-gate baseline circuit to two gates, cuts circuit depth from six to two, and reduces wall-clock execution time on real IBM hardware from 8.31 s to 6.06 s. The architecture deliberately mirrors the phases of a classical compiler — front end, optimization passes, and a delegating back end — which we believe makes the tool both understandable and extensible.

Future work proceeds along three lines. First, we will expand the rewrite library of the peephole pass and integrate a depth-aware and routing-aware cost model into A′ so that the exact search optimizes against the physical connectivity of the target device rather than gate count alone. Second, we will run the full benchmark suite on multi-qubit devices and publish aggregate statistics. Third, we will explore admissible heuristics for the A′ search beyond the zero heuristic, which we expect to extend the reach of the exact pass substantially.

X Acknowledgment

The author thanks the anonymous reviewers for comments that improved the presentation, and the IBM Quantum team for granting platform access used in the hardware experiments.

References

[1] F. Arute, K. Arya, R. Babbush, et al., "Quantum supremacy using a programmable superconducting processor," Nature, vol. 574, pp. 505–510, Oct. 2019.

[2] A. Aleksandrowicz, T. Alexander, P. Barkoutsos, et al., "Qiskit: An open-source framework for quantum computing," 2019. [Online]. Available: https://github.com/Qiskit/qiskit

[3] T. Toffoli, "Reversible computing," in Automata, Languages and Programming (Lecture Notes in Computer Science), Springer, 1980, pp. 632–644.

[4] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA, USA: Addison-Wesley, 2007.

[5] C. H. Bennett, "Logical reversibility of computation," IBM Journal of Research and Development, vol. 17, no. 6, pp. 525–532, 1973.

[6] S. S. Tannu and M. K. Qureshi, "Not all qubits are created equal: A case for variability-aware policies for NISQ-era quantum computers," in Proceedings of ASPLOS, 2019, pp. 987–999.

[7] F.-M. De Rainville, F.-A. Fortin, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," Journal of Machine Learning Research, vol. 13, pp. 2171–2175, 2012.

[8] Qiskit Development Team, "Qiskit transpiler documentation," 2024. [Online]. Available: https://docs.quantum.ibm.com

[9] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[10] A. Thota, "Quantum-Compiler: Hardware-aware quantum circuit synthesis," 2025. [Online]. Available: https://github.com/ThotaAvinash07-87/Quantum-Compiler